

Software Requirement Specification

for NexusTalk

Version 1.0

Author : Muthres Gurjar (2022UCS0097)

Table of Contents

1. Introduction

1.1. Purpose

1.2. Scope

1.3. Definitions, Acronyms, and Abbreviations

1.4. References

1.5. Document Overview

2. Overall Description

2.1. Product Perspective

2.2. Product Features Overview

2.3. User Classes and Characteristics

2.4. Operating Environment

2.5. Design and Implementation Constraints

3. System Features and Functional Specifications

3.1. Module 1: Core Communication Suite

3.2. Module 2: Community and Channel Infrastructure

3.3. Module 3: Privacy and User Control

3.4. Module 4: Smart & AI-Enhanced Features

3.5. Module 5: Moderation and Platform Safety

3.6 Core Foundational Features

4 External Interface Requirements

4.1. User Interfaces

4.2. Hardware Interfaces

4.3. Software Interfaces

5. Non-Functional Requirements

5.1. Performance

5.2. Security

5.3. Reliability

5.4. Usability

1. Introduction

1.1 Purpose

This document provides a complete and detailed specification of the requirements for the NexusTalk communication platform. It is the authoritative document that defines the system's scope, features, and behavior, serving as the foundational agreement between stakeholders and the engineering teams responsible for its design, development, and testing.

1.2 Scope

The scope of this project is the creation of a cross-platform messaging application that redefines digital communication. It encompasses secure one-to-one and group messaging, large-scale broadcasting via channels, unparalleled privacy controls, and AI-driven features to enhance user experience and efficiency. The initial release will target iOS, Android, and modern web browsers.

1.3 Definitions, Acronyms, and Abbreviations

- **ACL:** Access Control List
- **E2EE:** End-to-End Encryption
- **LLM:** Large Language Model
- **RBAC:** Role-Based Access Control
- **SFU:** Selective Forwarding Unit
- **SRS:** Software Requirement Specification

- **STT:** Speech-to-Text
- **UI/UX:** User Interface / User Experience
- **WebRTC:** Web Real-Time Communication

1.4 References

- Project Vision Document v1.0
- User Persona and Empathy Map Analysis v1.2

1.5 Document Overview

This SRS is organized into five primary sections. Section 1 provides this introduction. Section 2 offers a high-level description of the product and its operating context. Section 3 is the core of the document, providing a granular, functional specification for every system feature. Sections 4 and 5 define the system's external interfaces and critical non-functional requirements, respectively.

2. Overall Description

2.1 Product Perspective

NexusTalk is a self-contained, standalone application. It operates on a proprietary backend infrastructure and is not a plug-in or extension of any other software. It will interface with platform-level operating system services for push notifications, contact integration, and cloud storage for user-managed backups.

2.2 Product Features Overview

The platform is built around several key pillars:

- **Core Communication:** Secure text, voice, file sharing, and high-quality voice/video calls.
- **Community Building:** Advanced group management, public/private channels with roles, threaded discussions, and community folders.
- **Privacy Controls:** Default E2EE, granular profile visibility, ephemeral messaging, and screenshot notifications.
- **Smart Features:** AI-powered transcription, smart replies, and meeting summaries.
- **Platform Safety:** Robust reporting, user blocking, and automated content filtering in public spaces.

2.3 User Classes and Characteristics

- **The Everyday User:** Values simplicity and reliability for daily communication with family and friends.
- **The Privacy-Conscious User:** Prioritizes data security, encryption, and absolute control over their digital footprint.
- **The Community Manager:** Requires professional tools to manage large audiences, moderate discussions, and analyze engagement.
- **The Power User:** Seeks efficiency, multi-device synchronization, and advanced

customization options.

2.4 Operating Environment

- **Mobile Platforms:** iOS 16.0 or later, Android 9.0 (Pie) or later.
- **Web Platform:** Latest stable versions of Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge.

2.5 Design and Implementation Constraints

- All one-to-one and private group communications **must** implement the Signal Protocol for end-to-end encryption. This is a non-negotiable architectural foundation.
- The backend infrastructure must be designed on a scalable microservices architecture to ensure high availability and horizontal scaling.
- All user data stored on the client device must be encrypted at rest.
- The system must comply with all relevant international data protection regulations, including GDPR and CCPA.

3. System Features and Functional Specifications

This is where we use the precise, technical language from our document.

- **Functional Requirement:**
- **Non-Functional Requirement:**
 - "What it needs to do" = Functional Requirements
 - "How well it needs to do it" = Non-Functional Requirements

User Registration And Login

- **Purpose :** User must be able to register for the application through a valid phone number. On installing the application, the user must be prompted to register their phone number. If the user skips this step, the application should close. The user's phone number will be the unique identifier of his/her account on the platform.
- **Input** User provides a valid, active mobile phone number.
- **Processing** The system sends a One-Time Password (OTP) via SMS to the provided number and validates the code entered by the user.
- **Output** The user's account is successfully created, and they are granted access to the main interface of the application.

Adding New Contacts

- **Purpose** : The application should detect all contacts from the user's phone book upon being granted permission. If any of the contacts have user accounts with the application, those contacts must automatically be added to the user's contact list within the app. If any of the contacts have not yet registered, the user should be provided with an "Invite" option that can send those contacts a regular text message encouraging them to join, along with a link to the application's store page.
 - **Input** User grants the application permission to access the device's contact list.
 - **Processing** The application securely syncs the device's contacts, hashes the phone numbers, and cross
-

Feature Module:

Module 1: Core Chatting and Communication

1.1 Feature: Basic Messaging & Rich Media

Functional Requirements : -

- **1.1.1 Purpose of the Function:** "As an everyday user, I want to send text and voice messages, and use GIFs, stickers, and emojis so that I can have expressive conversations."
- **1.1.2 Inputs and Outputs:**
 - **Inputs:**
 - User-typed text (UTF-8 encoded string).
 - User-selected emoji, GIF, or sticker (Identifier from the respective service).
 - User-recorded audio data (Raw audio stream).
 - **Outputs:**
 - Rendered text message in the chat UI.
 - Displayed rich media (emoji, GIF, sticker) in the chat UI.
 - An audio player interface for the voice message.
 - Message status updates (Sent, Delivered, Read) sent to the sender.

Non-Functional Requirements

- **Performance:** Sending a text message should feel instant (less than 1 second delivery on

- a good connection). The emoji and GIF panel must load quickly.
- **Usability:** Recording a voice message should be simple and intuitive.

1.2 Feature: File Sharing

Functional Requirements :

- **1.2.1 Purpose of the Function:** "As an everyday user, I want to send photos, videos, and documents in my chats so that I can share important files and moments."
- **1.2.2 Inputs and Outputs:**
 - **Inputs:**
 - User-selected file from the device's storage, camera, or gallery.
 - `attachFile` command initiated by the user.
 - **Outputs:**
 - A message bubble in the chat containing a thumbnail (for images/videos) or file icon and name (for documents).
 - A progress bar indicating the upload/download status.
 - The actual file downloaded to the recipient's device upon request.

Non-Functional Requirements :-

How well it needs to do it:

- **Reliability:** Sending a large video file should not crash the app.
- **Performance:** File transfers should be as fast as the user's internet connection allows.

1.3 Feature: Voice and Video Calls

Functional Requirements :

- **1.3.1 Purpose of the Function:** To facilitate real-time, synchronous voice and video communication between two or more users, enabling more personal and immediate connection than text-based messaging.
- **1.3.2 Inputs and Outputs:**
 - **Inputs:**
 - User command to initiate a call (`startCall`).
 - Call type selection: `voice` or `video` (enum).
 - Target recipient(s) (User ID or Group ID).

- In-call user actions: `muteToggle`, `videoToggle`, `switchCamera`, `addParticipant`, `endCall`.
 - Live audio stream from the microphone.
 - Live video stream from the camera.
- **Outputs:**
 - An incoming call notification and UI on the recipient's device.
 - A dedicated call screen UI displaying participants, call duration, and control buttons.
 - Rendered video streams of participants.
 - Audible audio streams of participants.

Non-Functional Requirements

How well it needs to do it:

- **Performance:** Audio and video must be clear with minimal delay or lag.
- **Security:** All calls **must be** end-to-end encrypted.
- **Reliability:** Calls should not drop frequently on a stable internet connection.

1.4 Feature: Group Chats & Polls

Functional Requirements :

- **1.4.1 Purpose of the Function:** "As a user, I want to create groups with multiple members and create polls in a group so that we can all chat together and easily make decisions."
- **1.4.2 Inputs and Outputs:**
 - **Inputs (Group Creation):**
 - User command (`createGroup`).
 - List of initial members (array of User IDs).
 - Group name (string).
 - Group profile picture (image file, optional).
 - **Inputs (Poll Creation):**
 - User command (`createPoll`).
 - Poll question (string).
 - List of poll options (array of strings).
 - **Inputs (Voting):**
 - User command (`castVote`).
 - Poll ID and the selected Option ID.
 - **Outputs:**
 - A new group chat screen added to the users' chat lists.

- An interactive poll widget rendered within the group chat.
- Real-time updates to the vote counts and percentages displayed on the poll widget.

Non-Functional Requirements :

How well it needs to do it:

- **Usability:** The process of creating a group and a poll should be simple and take only a few taps.

1.5 Feature: Search and Customization

Functional Requirements :

- **1.5.1 Purpose of the Function:** "As a user, I want to search for information and customize the theme for each chat so that I can find things quickly and personalize my conversations."
- **1.5.2 Inputs and Outputs:**
 1. **Inputs (Search):**
 - Search query (string).
 - Search scope (current chat or all chats).
 2. **Inputs (Customization):**
 - Selected wallpaper (image file).
 - Selected theme color (hex code string).
 3. **Outputs (Search):**
 - A list of search results, each showing the message content, sender, and date.
 4. **Outputs (Customization):**
 - The chat background and UI elements are updated to reflect the new theme.

Non-Functional Requirements :

How well it needs to do it:

- **Performance:** Search results must appear very quickly, even if the user has thousands of messages.

1.6 Feature: Multi-Device & Personal Storage

Functional Requirements :

- **1.6.1 Purpose of the Function:** "As a power user, I want to use my account on multiple devices and have a "MySelf" chat so that I can continue my chats seamlessly and save notes for myself."
- **1.6.2 Inputs and Outputs:**
 1. **Inputs:**
 - Message sent from any linked device.
 - QR code scan from a new device.
 2. **Outputs:**
 - The sent message appears in near real-time on all other linked devices.
 - A "Saved Messages" or "MySelf" chat is present in the main chat list.

Non-Functional Requirements :

How well it needs to do it:

- **Reliability:** Messages must sync perfectly across all devices with no lost messages.
- **Performance:** The sync between devices should happen in near real-time (within a couple of seconds).

Feature: Privacy Tools (Disappearing Messages & Broadcasts)

Functional Requirements :

1.7 Feature: Privacy Tools (Disappearing Messages & Broadcasts)

- **1.7.1 Purpose of the Function:** "As a privacy-conscious user, I want to send messages that automatically delete, and as a user, I want to create broadcast lists to send a message to many people individually."
- **1.7.2 Inputs and Outputs:**
 1. **Inputs (Disappearing):**
 - Timer setting for a chat (e.g., 24 hours, 7 days, off).
 2. **Inputs (View Once):**
 - A "view once" toggle applied to a specific photo/video before sending.
 3. **Inputs (Broadcast):**
 - Creation of a broadcast list (list of User IDs).

- A message sent to the broadcast list.
- 4. **Outputs:**
 - Messages in a chat that automatically delete after the set time.
 - A message delivered as a standard, individual message to each person in the broadcast list.

Functional Requirements :

How well it needs to do it:

- **Security:** Disappearing messages **must be** permanently deleted from all devices and servers once their time is up.
 - **Usability:** It must be clear to the user that when they use a broadcast list, replies from recipients will come back in their private chat, not to the whole list.
-

Module 2: Channels and Communities

2.1 Feature: Channel Creation & Management

Functional Requirements :

- **2.1.1 Purpose of the Function:** "As a channel owner, I want to create public or private channels and assign roles like 'admin' with specific permissions so that I can safely manage my channel."
- **2.1.2 Inputs and Outputs:**
 - **Inputs:**
 - Channel details: `channel_name` (string), `channel_picture` (image file), `description` (string).
 - Channel type selection: `public` or `private` (enum).
 - Permission settings: `who_can_post` (enum: `all_members` or `only_admins`).
 - User command to `createChannel` or `updateChannelSettings`.
 - **Outputs:**
 - A newly created channel entity in the database.
 - A unique, cryptographically secure, and unguessable invite link (for private channels).
 - An updated channel settings UI reflecting the owner's choices.

Non-Functional Requirements :

How well it needs to do it:

- **Security:** Private channels **must not** be discoverable through public search. Their invite links must be unguessable.
 - **Reliability:** The permissions system must be strict. A regular member should never be able to post in a channel set to "Only Admins."
-

2.2 Feature: Content Management in Channels

Functional Requirements :

- **2.2.1 Purpose of the Function:** "As a channel owner, I want to upload documents and media, and also schedule posts to be published at a future time so that I can manage my content effectively."
- **2.2.2 Inputs and Outputs:**
 1. **Inputs:**
 - Message content (text, media file, poll, etc.).
 - User command to `scheduleMessage`.
 - Publication date and time (ISO 8601 timestamp).
 2. **Outputs:**
 - A confirmation message "Message scheduled for [Date] at [Time]".
 - A new entry in the "Scheduled Posts" management UI.
 - The actual message delivered to all channel members at the scheduled time.

Non-Functional Requirements :

How well it needs to do it:

- **Reliability:** Scheduled posts **must be sent** at exactly the chosen time, even if the admin who scheduled it is offline.
 - **Performance:** Admins should be able to upload large files to the channel efficiently.
-

2.3 Feature: Channel Analytics & Monetization

Functional Requirements :

- **2.3.1 Purpose of the Function:** "As a channel owner, I want to see analytics to

understand my audience, and as a content creator, I want to monetize my channel to earn an income."

- **2.3.3 Inputs and Outputs:**
 1. **Inputs (Monetization):**
 - Subscription price (decimal number).
 - Billing cycle (enum: `monthly`).
 - User's payment information (handled by the payment gateway's secure form).
 2. **Outputs (Analytics):**
 - Line charts and numerical data for member growth and post views.
 3. **Outputs (Monetization):**
 - Access granted to a private channel for a subscribed user.
 - Payouts transferred to the channel owner's linked bank account.

Non-Functional Requirements :

How well it needs to do it:

- **Performance:** The analytics dashboard must load quickly, even for very large channels.
 - **Security:** All payment processing and financial information **must be** handled with bank-grade security and encryption.
-

2.4 Feature: Community Engagement & Structure

Functional Requirements :

- **2.4.1 Purpose of the Function:** "As a community member, I want to reply to posts in threads and comment on them, and as an admin, I want to create communities with nested sub-groups to keep things organized."
- **2.4.2 Inputs and Outputs:**
 1. **Inputs:**
 - User command to `replyToMessage`.
 - The `message_id` of the parent message.
 - The content of the reply.
 - User command to `createCommunity` with a name and list of groups to include.
 2. **Outputs:**
 - A new message appearing in a threaded view, linked to its parent.
 - A new "Community" folder appearing in the user's chat list, containing the associated group chats.

Non-Functional Requirements :

How well it needs to do it:

- **Usability:** The design must be very clear so that users can easily navigate between the main channel, a comment thread, and the sub-groups of a community without getting lost.
-

2.5 Feature: User Tools (Broadcasts & Payments)

Functional Requirements :

Note: Broadcasts were detailed in the previous response and are a client-side function.

- **2.5.1 Purpose of the Function:** "As a user, I want to create a broadcast list to message many people at once, and I want to send payments to other members or groups."
- **2.5.2 Inputs and Outputs:**
 1. **Inputs:**
 - User command to `sendPayment`.
 - Payment amount (decimal number).
 - Currency (e.g., USD, EUR).
 - Recipient `user_id`.
 2. **Outputs:**
 - A special message bubble in the chat showing the payment amount and status.
 - A push notification to the recipient about the incoming payment.
 - Funds transferred to the recipient's account via the payment gateway.

Non-Functional Requirements :

How well it needs to do it:

- **Clarity (Usability):** For broadcasts, the app must make it clear that replies will come back to the sender privately, not to the whole list.
 - **Reliability & Security:** For payments, a transaction must either complete successfully or fail with a clear error message. It should never get stuck in an unknown state. All financial data **must be** encrypted.
-

Feature Module 3: Privacy and User Control

3.1 Feature: Total Encryption

Functional Requirements

- **3.1.1 Purpose of the Function:** "As a privacy-conscious user, I want all my communications and cloud backups to be end-to-end encrypted by default so that I know my entire history is completely private and secure."
- **3.1.2 Inputs and Outputs:**
 - **Inputs:**
 - Plaintext message content (string, file data, audio/video stream).
 - User command to `enableBackup`.
 - **Outputs:**
 - Encrypted ciphertext payload sent over the network.
 - An encrypted backup file (`.bkup`) stored in the user's cloud drive.
 - A 64-digit encryption key (string) displayed to the user for safekeeping.

Non-Functional Requirements :

How well it needs to do it:

- **Security (Top Priority!):** The end-to-end encryption **must be** on by default and cannot be turned off by the user. It needs to be state-of-the-art, ensuring that only the sender and recipient can ever see the content. This is a core promise of the app.
-

3.2 Feature: Profile, Status, and Search Privacy

Functional Requirements :

3.2.1 Purpose of the Function: "As a user, I want to control who sees my profile information and status, create a custom username, and control who can find me with it so that I can protect my identity."

3.2.2 Inputs and Outputs:

- **Inputs:**
 - User's selection for each privacy attribute (`profile_picture`, `status`, `about_info`, `username_search`): `Everyone`, `My Contacts`, `Nobody` (enum).

- An exception list of `user_ids` for the "My Contacts Except..." option.
- A unique `username` (string).
- **Outputs:**
 - An updated record in the user's privacy settings on the server.
 - When other users view the profile, they see either the user's data or a default placeholder, according to the rules.

Non-Functional Requirements :

How well it needs to do it:

- **Reliability:** These privacy rules **must be** strictly enforced. If a user sets their picture to "Nobody," there can be no bugs that would ever allow a stranger to see it.
-

3.3 Feature: Disappearing Media Controls

Functional Requirements :

- **3.3.1 Purpose of the Function:** "As a privacy-conscious user, I want to be notified if someone screenshots my disappearing photo, and I want to prevent others from downloading the media I send."
- **3.3.2 Inputs and Outputs:**
 1. **Inputs:**
 - An OS-level event indicating a screenshot was taken.
 - The sender's choice to "Block Downloads" (`true/false`) on a "View Once" message.
 2. **Outputs:**
 - A system notification message delivered to the sender: "[Recipient's Name] took a screenshot."
 - The absence of a "Save to gallery" button in the recipient's media viewer UI.

Non-Functional Requirements :

How well it needs to do it:

- **Responsiveness:** The screenshot notification must be sent instantly.
 - **Security:** The "Block Downloads" feature must effectively prevent the recipient from using the app's interface to save the file to their device.
-

3.4 Feature: Chat Management & Organization

Functional Requirements :

- **3.4.1 Purpose of the Function:** "As a user, I want to mute, pin, archive, and star messages so that I can keep my chat list organized and easily find important things."
- **3.4.2 Inputs and Outputs:**
 1. **Inputs:**
 - User commands initiated by long-pressing a chat or message: `pinChat`, `muteChat`, `archiveChat`, `starMessage`.
 2. **Outputs:**
 - An immediate visual update to the chat list or message UI.
 - An updated record in the local metadata database.

Non-Functional Requirements :

How well it needs to do it:

- **Usability:** All these actions should be easy to find, fast to use, and simple to undo.
-

3.5 Feature: Advanced Message & Presence Control

Functional Requirements :

3.5.1 Purpose of the Function: "As a user, I want to delete messages for everyone and go into 'stealth mode' so that I have complete control over my conversations and online presence."

3.5.2 Inputs and Outputs:

- **Inputs:**
 - User command `deleteMessage` with `scope: 'everyone'`.
 - An on/off toggle for "Stealth Mode".
 - A selected date for the "Clear History" function.
- **Outputs:**
 - A `recall_message` command broadcast to relevant clients.
 - An updated presence visibility setting on the server.
 - Messages before a certain date are deleted from the local device's storage.

Non-Functional Requirements :

How well it needs to do it:

- **Reliability:** "Delete for Everyone" must reliably and quickly remove the message from all recipient devices. When "Stealth Mode" is on, the user's "online" or "last seen" status **must never** be shown to anyone.
-

Module 4: Privacy and User Control

Feature: Voice-to-Text Transcription

Functional Requirements :

- **4.1.1 Purpose of the Function:** "As an everyday user, I want to turn voice notes into text **so that** I can read a message in a quiet place (like a library or meeting) without listening to it out loud."
- **4.1.2 Inputs and Outputs:**
 - **Inputs:**
 - User command to `transcribeMessage`.
 - The audio file of the voice message (e.g., in Opus or AAC format).
 - The `message_id` of the voice note.
 - **Outputs:**
 - A text string containing the transcription of the voice message.
 - A state change in the UI, displaying the text below the corresponding voice message player.

Non-Functional Requirements :

How well it needs to do it:

- **Accuracy:** The transcription must be highly accurate, correctly identifying at least 95% of the words in a clear recording.
 - **Performance:** The text transcription should appear quickly, ideally within a few seconds of the request.
-

Feature: Smart Reply Suggestions

Functional Requirements :

- **4.2.1 Purpose of the Function:** "As a busy user, I want to see smart reply suggestions **so that** I can respond to common messages quickly with a single tap."
- **4.2.2 Inputs and Outputs:**
 1. **Inputs:**
 - An event trigger: `newMessageReceived`.
 - The text content of the newly received message and the preceding 1-2 messages for context (an array of strings).
 2. **Outputs:**
 - An array of 3 short strings, representing the suggested replies.
 - Three tappable buttons displayed in the UI above the text input area.

Non-Functional Requirements :

How well it needs to do it:

- **Relevance:** The suggestions must be contextually appropriate for the conversation.
 - **Performance:** The reply suggestions must appear almost instantly after a new message is received.
-

Feature: Smart Media Search

Functional Requirements :

- **4.3.1 Purpose of the Function:** "As a user, I want to search my media using keywords (like 'dog', 'beach', or text from a screenshot) **so that** I can find a specific photo or video quickly."
- **4.3.2 Inputs and Outputs:**
 1. **Inputs:**
 - An image or video file.
 - A user's search query (string).
 2. **Outputs:**
 - A set of keyword tags associated with a media file in the local database.
 - A grid of media thumbnails that match the user's search query.

Non-Functional Requirements :

How well it needs to do it:

- **Accuracy:** The object and text recognition must be accurate enough to provide useful results.
- **Privacy:** This scanning and indexing process **must** happen on the user's device only, not on the company's servers, to protect user privacy.

- **Performance:** Search results should appear within a few seconds.
-

Feature: AI Meeting Summaries

Functional Requirements :

- **4.4.1 Purpose of the Function:** "As a professional, I want to get an AI-generated summary of recorded group calls **so that** I can quickly catch up on meetings I missed or review key decisions."
- **4.4.2 Inputs and Outputs:**
 1. **Inputs:**
 - User command to **generateSummary**.
 - A multi-channel audio file from a recorded group call.
 - Consent from all participants to record and process the call.
 2. **Outputs:**
 - A structured text summary, formatted with headings like "Key Topics," "Decisions," and "Action Items."

Non-Functional Requirements :

How well it needs to do it:

- **Accuracy:** The summary must accurately reflect the main points of the conversation.
 - **Clarity:** The generated summary must be well-structured and easy to read.
-

Feature Module 5: Moderation and Reporting

Feature: Reporting & Context

Functional Requirements :

- **5.1.1 Purpose of the Function:** "As a user, I want to report specific content or users with clear reasons and include recent messages **so that** the moderation team has the full context to make a fair decision."
- **5.1.3 Inputs and Outputs:**
 1. **Inputs:**

- User command to `createReport`.
- `target_id`: The unique identifier of the message, user, or group being reported.
- `reason_code`: An enum representing the user's selected reason (e.g., `spam`, `harassment`, `hate_speech`).
- `include_context`: A boolean (`true/false`) indicating if the user consented to include recent messages.

2. Outputs:

- A new ticket created in the moderation system's database.
- A success message displayed to the user: "Report submitted. Thank you for helping keep our community safe."

Non-Functional Requirements :

How well it needs to do it:

- **Usability:** The reporting process should be quick and easy to find, so users are encouraged to report bad behavior.
 - **Privacy:** The reported messages sent for context must be handled securely and only be visible to the authorized moderation team.
-

Feature: User Blocking

Functional Requirements :

5.2.1 Purpose of the Function: "As a user, I want to block another person **so that** I can immediately stop them from contacting me or seeing my profile."

5.2.2 Inputs and Outputs:

- **Inputs:**
 - User command to `blockUser` or `unblockUser`.
 - The `user_id` of the user to be blocked/unblocked.
- **Outputs:**
 - A new row inserted into (or removed from) the `block_list` table.
 - An immediate cessation of all interactions between the two users.
 - The blocked user is removed from the blocker's contact list and vice versa.

Non-Functional Requirements :

How well it needs to do it:

- **Reliability:** The block **must be** 100% effective and take effect instantly. There can be no bugs that allow a blocked user's messages to get through.
-

Feature: Automated Content Filtering

Functional Requirements :

- **5.3.1 Purpose of the Function:** "As a user, I want the platform to automatically filter spam, scams, and hate speech **so that** my environment is safer and free from harmful content."
- **5.3.2 Inputs and Outputs:**
 1. **Inputs:**
 - Content (text or image file) posted to a public channel.
 2. **Outputs:**
 - A moderation status for the content: **approved**, **quarantined**, or **rejected**.
 - **Quarantined** content is hidden from public view but flagged for human review.

Non-Functional Requirements :

How well it needs to do it:

- **Accuracy:** The filter should be very good at catching genuinely harmful content while having a very low rate of incorrectly flagging normal, safe content (false positives).
 - **Performance:** This filtering process must happen in the background and not slow down the delivery of messages for the user.
-

Feature: Parental Controls

Functional Requirements :

- **5.4.1 Purpose of the Function:** "As a parent, I want to enable parental controls for my child's account **so that** I can restrict their exposure to inappropriate content and ensure their safety."
- **5.4.2 Inputs and Outputs:**
 1. **Inputs:**
 - A link request from a parent account to a child account.
 - An explicit acceptance of the link from the child account.
 - The parent's desired safety settings (e.g., **contact_policy**).

`'contacts_only')`.

2. **Outputs:**

- A secure, consent-based link between the parent and child accounts in the database.
- The child's account settings are updated to reflect the parent's choices.

Non-Functional Requirements :

How well it needs to do it:

- **Security:** The process for linking accounts must be secure to prevent unauthorized access. The child should not be able to easily disable the controls set by the parent.
-

Feature: Automated Sanctions

Functional Requirements :

- **5.5.1 Purpose of the Function:** "As a community member, I want the platform to automatically issue temporary bans to users who repeatedly break the rules **so that** offenders are dealt with quickly and the community remains safe."
- **5.5.2 Inputs and Outputs:**
 1. **Inputs:**
 - A `add_strike` command issued by an authorized human moderator to a specific `user_id`.
 2. **Outputs:**
 - An updated account status for the user (e.g., `active`, `suspended`, `banned`).
 - A `suspension_end_time` timestamp set for temporary bans.
 - A system-generated notification sent to the user explaining the sanction, its duration, and the reason.

Non-Functional Requirements :

How well it needs to do it:

- **Fairness & Reliability:** The automated system must be highly reliable, only issuing bans based on confirmed violations to avoid unfairly punishing innocent users.

3.6 Core Foundational Features

- **Registration and Login (User Accounts)**

- **Basic Messaging and Rich Media**
- **File Sharing**
- **Voice and Video Calls**
- **Group Chats**
- **End-to-End Encryption**

These are the features without which the app cannot fulfill its primary purpose of enabling communication between users.

While other functionalities like channels, smart replies, and customization enhance the user experience, the app would be non-functional and unable to launch without the core capabilities of sending messages, sharing files, making calls, forming groups, and ensuring user privacy through encryption.

4. External Interface Requirements

4.1 User Interfaces

The application shall present a clean, intuitive, and consistent user interface across all platforms. It must be fully responsive and support both light and dark themes, adhering to platform-specific human interface guidelines.

4.2 Hardware Interfaces

The application will interface with the device's camera, microphone, local storage, and GPS/location services.

4.3 Software Interfaces

The application will interface with:

- **OS APIs:** For contact list integration, push notifications (APNS, FCM), and native file pickers.
- **Payment Gateway APIs:** For handling channel subscriptions.
- **Cloud STT and LLM APIs:** For transcription and summarization features.

5. Non-Functional Requirements

5.1 Performance

- **Message Latency:** 99% of text messages must be delivered in under 1.5 seconds under nominal network conditions.
- **App Launch Time:** The application must achieve a cold start to an interactive state in under 3 seconds.
- **Call Quality:** Voice and video calls must maintain a high Mean Opinion Score (MOS) of >4.0 on a stable broadband connection.

5.2 Security

- The implementation of the Signal Protocol shall be subject to a third-party security audit prior to launch.
- The system must be protected against common vulnerabilities, including MITM attacks, SQL injection, and XSS.
- All APIs must require authentication and authorization.

5.3 Reliability

- The service shall maintain a minimum uptime of 99.95%.
- The system must guarantee zero message loss during transit.
- The client application must handle network interruptions gracefully and reconnect automatically.

5.4 Usability

- The new user registration and onboarding process must be completed in under 90 seconds.
- The application must comply with WCAG 2.1 AA accessibility standards.

5.5 Maintainability

- The codebase must be modular, well-documented, and adhere to established coding standards for each platform.
- The system must have comprehensive logging, monitoring, and alerting to facilitate proactive maintenance.